

VGG16 FEATURE EXTRACTOR + SVM

GÖZ BÖLGESİ DEEPPFAKE DENEYİ

Kod Açıklama ve Teknik İşleyiş Raporu

Raporun amacı

Bu doküman, notebook içindeki kodu hücre ve bileşen bazında açıklar. Arkadaşların kodu açmadan önce deneyin hangi model ve yöntemleri kullandığını, verinin nasıl işlendiğini, model seçiminin nasıl yapıldığını ve hangi güvenlik/kalite kontrollerinin uygulandığını anlayabilsin diye hazırlanmıştır.

Alan	Bilgi
Kaynak notebook	Kader_Deney1_Goz_VGG16_SVM_SSOT_v2_FIXED_v3 (1).ipynb
Deney bölgesi	Göz - Combined Eye ROI
Ana yaklaşım	VGG16 derin özellik çıkarımı + SVM sınıflandırma
Model tipi	Hibrit: Önceden eğitilmiş CNN feature extractor + klasik makine öğrenmesi sınıflandırıcısı
Pozitif sınıf	FAKE = 1
Negatif sınıf	REAL = 0
Kod standardı	SSOT v2.0 uyumlu deney pipeline'ı
Rapor tarihi	8 Ağustos 2026

İçindekiler

1. Deneyin Amacı ve Genel Yaklaşım
2. Kullanılan Model ve Model Türü
3. Uçtan Uca Veri Akışı
4. Çalışma Ortamı, Seed ve Bağımlılıklar
5. Drive, Run ID ve Çıktı Klasör Yapısı
6. Veri Kaynağı ve Canonical Metadata
7. Veri Muhasebesi ve Kalite Kapıları
8. SHA-256, Veri Bütünlüğü ve Split Sızıntısı Kontrolü
9. VGG16 Feature Extractor
10. Görüntü Ön İşleme ve Feature Extraction
11. Smoke Test Mantığı
12. SVM Hiperparametre Seçimi
13. Final Train+Validation Eğitimi ve Atomik Model Kaydı
14. Test Değerlendirmesi ve Metrikler
15. Video-Level Değerlendirme
16. Grafik ve Görsel Çıktılar
17. Tekrarlanabilirlik ve Teslim Dosyaları
18. Kodun Güçlü Yönleri
19. Teknik Sınırlamalar ve Dikkat Noktaları
20. Özet Pseudocode

1. Deneyin Amacı ve Genel Yaklaşım

Notebookun amacı, göz bölgesinden daha önce çıkarılmış combined-eye ROI görüntülerini REAL ve FAKE olarak sınıflandırmaktır. Kod doğrudan bir CNN'i göz veri seti üzerinde baştan sona eğitmez. Bunun yerine ImageNet üzerinde önceden eğitilmiş VGG16 ağı sabit bir derin özellik çıkarıcı olarak kullanılır; çıkarılan sayısal özellikler SVM'e verilir.

Combined Eye ROI

- > VGG16 (ImageNet, include_top=False, pooling='avg', frozen)
- > 512-D feature vector
- > StandardScaler
- > SVM
- > REAL / FAKE

Bu deney nasıl adlandırılabilir?

Bu yapı saf bir CNN fine-tuning deneyi değildir. En doğru tanım, "deep feature extraction + classical machine learning classification" yani "derin özellik çıkarımı + klasik makine öğrenmesi sınıflandırması" şeklindedir.

2. Kullanılan Model ve Model Türü

Bileşen	Tür	Kodda Kullanım Şekli
VGG16	Convolutional Neural Network (CNN)	ImageNet pretrained, sınıflandırma başlığı çıkarılmış, tamamen dondurulmuş feature extractor
Global Average Pooling	Özellik sıkıştırma	Son convolutional feature map'leri 512 boyutlu tek vektöre dönüştürür
StandardScaler	Ölçekleme / normalizasyon	SVM'e girecek feature boyutlarını ortalama 0 ve standart sapma 1 ölçeğine getirir
SVC / SVM	Klasik supervised machine learning sınıflandırıcısı	Linear ve RBF kernel adayları validation üzerinde karşılaştırılır

2.1 VGG16'nın rolü

VGG16 bu deneyin son karar veren modeli değildir. `include\_top=False` kullanıldığı için orijinal ImageNet sınıflandırma katmanları kaldırılır. `pooling='avg'` ile son convolutional çıktılar Global Average Pooling üzerinden 512 boyutlu feature vektörüne dönüştürülür. `trainable=False` olduğu için VGG16 ağırlıkları göz verisi üzerinde güncellenmez.

2.2 SVM'nin rolü

Asıl REAL/FAKE sınıflandırma kararı SVM tarafından verilir. VGG16'nın ürettiği 512 boyutlu özellikler önce StandardScaler ile ölçeklenir, ardından SVC sınıflandırıcısına girer. Bu nedenle sistem iki aşamalı hibrit bir modelleme yaklaşımıdır.

3. Uçtan Uca Veri Akışı

Aşama	Ne Yapılıyor?	Çıktı
1. Veri okuma	metadata.csv okunur, başarılı	Canonical metadata / inventory

	combined-eye kayıtları belirlenir.	
2. Veri doğrulama	Şema, sample ID, dosya yolu, sınıf ve split kontrolleri yapılır.	Quality gate sonuçları
3. Hashing	ROI görüntülerinin SHA-256 değerleri hesaplanır; işlem resume edilebilir.	sha256_progress.csv
4. Leakage kontrolü	Aynı içerik farklı splitlerde mi ve source_video kesişimi var mı kontrol edilir.	Leakage audit
5. Feature extraction	224x224 RGB görüntüler VGG16 preprocess_input sonrası VGG16'dan geçirilir.	512-D feature matrisleri
6. Model seçimi	Linear/RBF SVM adayları train üzerinde fit edilir, validation üzerinde sıralanır.	En iyi SVM parametreleri
7. Final model	Train+val feature'ları birleştirilir ve final SVM eğitilir.	best_model.joblib + scaler
8. Test	Test seti yalnızca final aşamada değerlendirilir.	Metrikler ve tahmin CSV'leri
9. Raporlama	Grafikler, environment, manifest ve run summary oluşturulur.	Run çıktıları

4. Çalışma Ortamı, Seed ve Bağımlılıklar

İlk kod hücresi gerekli Python paketlerini kurar ve TensorFlow/Keras, scikit-learn, NumPy, pandas, PIL, matplotlib, PyYAML ve joblib bileşenlerini yükler. Tekrarlanabilirlik için Python hash seed, Python random, NumPy ve TensorFlow seed değeri 42 yapılır.

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
```

Kütüphane	Görevi
TensorFlow / Keras	VGG16'yı yükleme ve feature extraction
scikit-learn	StandardScaler, SVC ve değerlendirme metrikleri
PIL	ROI görüntülerini açma, RGB dönüşümü ve resize
NumPy	Feature matrisleri ve sayısal kontroller
pandas	Metadata, audit ve CSV çıktıları
joblib	Final SVM ve scaler nesnelerini kaydetme/yükleme
hashlib	SHA-256 içerik hash'i ve deterministik ID üretimi
matplotlib	Confusion Matrix, ROC ve Precision-Recall grafikleri

5. Drive, Run ID ve Çıktı Klasör Yapısı

Notebook Google Colab için tasarlanmıştır. Drive bağlandıktan sonra AISC proje kökü, Deneyler, Kader, Deney 1, Göz ve `eye\_roi\_output` dizinleri Unicode normalizasyonu kullanılarak güvenli şekilde çözülür. Böylece Türkçe karakterlerin Drive üzerinde farklı normalization biçimlerinde görünmesi nedeniyle oluşabilecek yol problemleri azaltılır.

5.1 Benzersiz RUN_ID

```
RUN_ID = YYYYMMDD_HHMM_eye_vgg16_svm_seed42
```

Her çalıştırma farklı bir RUN_ID alır. Aynı RUN_ID klasörü zaten varsa kod `FileExistsError` üretir. Böylece iki farklı deney aynı klasöre yazılmaz.

5.2 Run klasörü

```
Sonuçlar/outputs/<RUN_ID>/
├── checkpoints/
├── logs/
├── metrics/
├── predictions/
├── figures/
└── artifacts/
```

Kaynak `eye\_roi\_output` veri dizini salt okunur politika ile kullanılır. Kod kaynak ROI'leri silmez, taşımaz veya yeniden adlandırmaz.

6. Veri Kaynağı ve Canonical Metadata

Ana veri kaynağı `eye\_roi\_output/metadata.csv` dosyasıdır. Notebook klasörleri körlemesine taramak yerine metadata üzerinden hangi ROI'nin hangi sınıfa ve split'e ait olduğunu takip eder.

6.1 Kaynakta zorunlu alanlar

```
SOURCE_REQUIRED = {"label", "split", "status", "combined_eye_path"}
```

Bu sütunlardan biri yoksa schema quality gate başarısız olur ve deney başlamaz. Split isimlerinde `validation` ve `valid` değerleri `val` olarak normalize edilir.

6.2 Canonical SSOT alanları

Alan	Anlamı
sample_id	Canonical ROI satırı için deterministik ve benzersiz kimlik
source_sample_id	Kaynak metadata'daki orijinal sample_id; tekrar edebilir
source_video	Varsa orijinal video kimliği / adı
frame_index	Varsa frame numarası
face_index	Aynı frame içindeki yüz indeksi
roi_state	Varsa göz/ROI durum bilgisi
label	real veya fake
split	train / val / test

status	SUCCESS / SKIPPED / ERROR
skip_reason	Varsa atlama sebebi
sha256	ROI görüntüsünün içerik hash'i
output_path	Combined-eye ROI dosya yolu
run_id	Bu canonical kaydı oluşturan run kimliği

6.3 Duplicate sample_id problemi nasıl çözüldü?

Kaynak metadata'daki `sample\_id` değerinin ROI satırı seviyesinde benzersiz olduğu varsayılmaz. Aynı kaynak frame altında birden fazla yüz veya ROI bulunabileceği için kaynak ID ayrı bir `source\_sample\_id` alanında korunur. Canonical `sample\_id`, source ID + video/frame/face + label + split + ROI path bileşiminden SHA-256 ile deterministik biçimde üretilir. Bu yöntem yalnızca rastgele bir sıra numarası ekleyerek problemi gizlemez; gerçek duplicate ROI kayıtlarının tespit edilebilmesini korur.

7. Veri Muhasebesi ve Kalite Kapıları

Kodun önemli tasarım özelliklerinden biri, sadece 'çalıştı' demek yerine her kritik aşamada quality gate kullanmasıdır. Bir gate başarısız olursa süreç durur.

```
total_inputs == success_count + skipped_count + error_count
canonical sample_id unique olmalı
her split boş olmamalı
her split REAL ve FAKE içermeli
SUCCESS görüntü dosyası diskte bulunmalı
```

Neden önemli?

Modelin yüksek skor üretmesi tek başına güvenilirlik anlamına gelmez. Hangi görüntülerin kullanıldığı, neyin atlandığı ve splitlerin doğru olduğu doğrulanmadan performans metrikleri bilimsel olarak yanıltıcı olabilir.

8. SHA-256, Veri Bütünlüğü ve Split Sızıntısı Kontrolü

8.1 SHA-256 içerik hash'i

Her SUCCESS combined-eye görüntüsünün SHA-256 değeri hesaplanır. Aynı dosya içeriği farklı isimle kopyalanmış olsa bile hash aynı olacağı için path tabanlı kontrole göre daha güçlü bir duplicate/leakage kontrolü sağlar.

8.2 Drive-safe ve resumable hashing

Mekanizma	Kodda Ne Sağlıyor?
Retry	Dosya erişimi geçici olarak başarısız olursa 5 kez yeniden denir.
Path rebuild	Eski/stale Drive absolute path'i mevcut EYE_ROI_ROOT altında yeniden kurmaya çalışır.
Hedefli dosya arama	Tüm Drive yerine yalnız ilgili class/split/combined dizininde dosya adına bakar.
Progress save	Her 50 hash'te `sha256_progress.csv` atomik olarak güncellenir.

Resume	Hücre tekrar çalıştırılırsa tamamlanan hash'ler yeniden hesaplanmaz.
Missing audit	Gerçekten erişilemeyen dosyalar `missing_or_unreadable_images.csv` içine yazılır ve süreç durur.

8.3 Exact-content split leakage

Aynı SHA-256 değeri birden fazla splitte görülürse kod `cross\_split\_content\_leakage.csv` üretir ve deneyi durdurur. Bu kontrol, aynı ROI içeriğinin hem train hem validation/test tarafına sızmasını önlemeyi amaçlar.

8.4 Source-video izolasyonu

Eğer `source\_video` alanı bütün örneklerde güvenilir şekilde mevcutsa train, validation ve test video kümelerinin birbirinden ayrık (`isdisjoint`) olduğu doğrulanır. Source-video bilgisi yoksa kod bu bilgiyi tahmin etmez; gate sonucu `NOT\_VERIFIABLE` olarak kaydedilir.

9. VGG16 Feature Extractor

Parametre	Kod Değeri	Anlamı
architecture	VGG16	CNN tabanlı derin özellik çıkarıcı
weights	imagenet	ImageNet üzerinde önceden öğrenilmiş ağırlıklar
include_top	False	Orijinal 1000 sınıflı classification head kullanılmaz
pooling	avg	Global Average Pooling
trainable	False	VGG16 ağırlıkları güncellenmez
input	224 x 224 x 3	RGB göz ROI
feature_dim	512	Her görüntü için üretilen derin feature sayısı

Bu seçim nedeniyle VGG16, göz bölgesinde manipülasyonu doğrudan sınıflandırmak yerine görüntüyü 512 sayılı kompakt bir temsil haline getirir. Öğrenme yükü SVM aşamasına taşınır.

10. Görüntü Ön İşleme ve Feature Extraction

Her ROI PIL ile açılır, RGB'ye çevrilir ve 224x224 boyutuna bilinear interpolation ile yeniden boyutlandırılır. Sonra NumPy float32 dizisine çevrilir ve Keras VGG16 `preprocess\_input` uygulanır.

```
img = img.convert('RGB')
img = img.resize((224, 224), Image.Resampling.BILINEAR)
x = preprocess_input(np.stack(arrays, axis=0))
features = feature_extractor.predict(x)
```

Batch size 32'dir. Her feature batch'inin iki boyutlu olması ve ikinci eksenin tam olarak 512 olması zorunludur. Feature matrisinde NaN veya Inf bulunursa numerical quality gate süreci durdurur.

10.1 Feature cache yaklaşımı

Bu SSOT sürümünde run klasörleri immutable kabul edildiği için eski run'dan feature cache yeniden kullanılmaz. Her run kendi train/val/test NPZ feature dosyalarını yeniden üretir. Bu yaklaşım çalışma süresini artırabilir; karşılığında eski bir cache'in yanlış deneye karışma riskini azaltır.

11. Smoke Test Mantığı

Tam feature extraction başlamadan önce küçük bir kalite testi uygulanır. Yaklaşık iki batch train örneği VGG16'dan geçirilir; feature boyutları ve sayısal değerler kontrol edilir. Ardından küçük bir Linear SVM fit/predict testi yapılır.

Smoke Gate	Kontrol
VGG16 forward	Görüntüler feature extractor'dan hatasız geçiyor mu?
Feature shape	Çıktı gerçekten N x 512 mi?
Numerical	NaN / Inf var mı?
Mini-SVM fit	Feature'lar ile SVM fit çalışıyor mu?
Mini-SVM predict	Model tahmin üretebiliyor mu?

Backward	NOT_APPLICABLE - VGG16 frozen ve gradient tabanlı eğitim yapılmıyor
----------	---

12. SVM Hiperparametre Seçimi

12.1 Ölçekleme

Model seçimi sırasında StandardScaler yalnızca train feature'ları üzerinde `fit\_transform` edilir. Validation tarafında sadece `transform` uygulanır. Böylece validation dağılımı scaler parametrelerini öğrenmek için kullanılmaz.

12.2 Arama uzayı

Kernel	C	Gamma
Linear	0.1, 1.0, 10.0	scale
RBF	1.0, 10.0, 100.0	scale, 0.001, 0.01

12.3 En iyi modelin seçimi

Her SVM adayı train üzerinde fit edilir ve validation üzerinde tahmin edilir. Adaylar sırayla `val\_f1`, `val\_roc\_auc` ve `val\_accuracy` metriklerine göre azalan biçimde sıralanır. İlk satır final model parametreleri olarak seçilir.

```
selection_sort = ['val_f1', 'val_roc_auc', 'val_accuracy']
best_params = search_df.iloc[0]
```

Test seti model seçimine katılmaz

Kodun önemli metodolojik kuralı budur. Test verisi kernel, C, gamma veya model seçimi için kullanılmaz; yalnızca final model hazır olduktan sonra değerlendirme aşamasında açılır.

13. Final Train+Validation Eğitimi ve Atomik Model Kaydı

En iyi SVM parametreleri belirlendikten sonra train ve validation feature'ları birleştirilir. Yeni bir StandardScaler train+validation birleşik veri üzerinde fit edilir. Test seti yalnızca bu scaler ile transform edilir.

```
X_trainval = concatenate(X_train, X_val)
final_scaler.fit_transform(X_trainval)
X_test_scaled = final_scaler.transform(X_test)
```

Final SVC `probability=True` ile oluşturulur. Böylece yalnız sınıf etiketi değil, FAKE sınıfı için olasılık değeri de üretilebilir.

13.1 Kaydedilen final dosyalar

Dosya	İşlev
best_model.joblib	Final SVM sınıflandırıcısı
best_scaler.joblib	Train+validation üzerinde fit edilmiş StandardScaler
best_state.json	Run ID, seçilen parametreler, seed, sample sayıları ve fit süresi

13.2 Atomik kayıt

Model ve scaler önce `.tmp` dosyasına yazılır, tekrar yüklenerek doğrulanır ve ardından `os.replace` ile hedef dosyanın yerine geçirilir. Bu mekanizma, kayıt sırasında sistem/Drive kesintisi olursa yarım ve bozuk final model dosyası bırakma riskini azaltır.

13.3 Fresh reload inference testi

Kaydedilen model ve scaler diskten yeniden yüklenir. Test setinden küçük bir örnek üzerinde orijinal model ve yeniden yüklenen modelin tahminlerinin aynı olması beklenir. Uyuşmazlık varsa inference quality gate başarısız olur.

14. Test Değerlendirmesi ve Metrikler

Final test aşamasında diskten yüklenen model kullanılır. Sınıf tahmini, `predict\_proba` ve `decision\_function` değerleri alınır. FAKE sınıfının probability sütunu `classes\_` üzerinden dinamik olarak bulunur.

Metrik	Kodda Anlamı
Accuracy	Tüm test frame'leri içinde doğru tahmin oranı
Precision	FAKE tahminlerinin ne kadarının gerçekten FAKE olduğu
Recall	Gerçek FAKE örneklerin ne kadarının yakalandığı
F1-score	Precision ve recall dengesini özetleyen harmonik ortalama
ROC-AUC	FAKE olasılık skorlarının sınıfları ayırma gücü
Average Precision	Precision-Recall eğrisinin özet değeri
Classification Report	REAL ve FAKE için precision, recall, F1 ve support
Confusion Matrix	True/Pred REAL-FAKE sayımlarının 2x2 matrisi

14.1 Test prediction CSV

`test\_predictions.csv` içinde sample_id, source_video, image_path, gerçek etiket, tahmin edilen etiket, FAKE olasılığı, decision score ve run_id saklanır. Böylece tek tek hatalı tahminler daha sonra analiz edilebilir.

Bu rapor sonuç raporu değildir

Notebook dosyası kod mantığını içeriyor; bu doküman da kod açıklamasına odaklanıyor. Burada belirli accuracy/F1 değerleri verilmemesinin nedeni, güvenilir final performansın ancak notebook tamamlandıktan sonra oluşan run çıktılarından okunabilmesidir.

15. Video-Level Değerlendirme

Video-level değerlendirme yalnız `source\_video` split quality gate gerçekten PASSED olduğunda yapılır. Aynı videoya ait framelerin FAKE olasılıkları ortalananır. Ortalama olasılık 0.5 veya üzerindeyse video FAKE, altındaysa REAL kabul edilir.

```
video_fake_probability = mean(frame_fake_probabilities)
predicted_label = 1 if probability >= 0.5 else 0
```

Kaynak metadata'da gerçek video kimliği güvenilir biçimde mevcut değilse kod video-level metrik üretmez. Bu, hatalı bir video gruplamasını performans sonucu gibi raporlamamak için bilinçli bir güvenlik kararıdır.

16. Grafik ve Görsel Çıktılar

Grafik	Açıklama
Confusion Matrix	REAL/FAKE doğru ve yanlış tahmin sayılarını gösterir
ROC Curve	False Positive Rate - True Positive Rate ilişkisini ve ROC-AUC'yi gösterir
Precision-Recall Curve	Recall - Precision ilişkisini ve Average Precision değerini gösterir

Grafikler 10x6 inch, 150 DPI olarak kaydedilir. Başlıklar, eksenler ve legend metinleri İngilizcedir. Kaydedilen her görüntü tekrar açılır ve kısa kenarın en az 600 piksel olduğu kodla doğrulanır.

17. Tekrarlanabilirlik ve Teslim Dosyaları

Dosya/Klasör	İçerik
config_resolved.yaml	Çalıştırmada kullanılan tüm çözülmüş ayarlar
logs/training.log	Run akışı ve önemli bilgilendirmeler
artifacts/metadata_canonical_ssot.csv	Canonical metadata manifest
artifacts/sha256_progress.csv	Resume edilebilir SHA-256 ilerleme kaydı
artifacts/quality_gate_audit.json	Şema, hash ve split kontrolleri
artifacts/smoke_test.json	Smoke test sonucu
artifacts/checkpoint_test.json	Atomik model kaydı ve reload test sonucu
artifacts/figure_quality_audit.json	Görsel kalite doğrulaması
artifacts/*_features.npz	Train/val/test VGG16 feature matrisleri
metrics/svm_validation_search.csv	Tüm SVM adaylarının validation sonuçları
metrics/metrics.json	Final metriklerin ve quality gate durumlarının ana özeti
predictions/test_predictions.csv	Örnek bazlı test tahminleri
environment.json	Python, platform, TensorFlow, NumPy, pandas, sklearn, GPU ve seed
requirements_lock.txt	pip freeze ile paket sürümleri
run_summary.json	Makine tarafından okunabilir final run özeti
output_manifest.csv	Tüm run dosyalarının boyutu ve SHA-256 özeti

18. Kodun Güçlü Yönleri

- VGG16 ve SVM rolleri açık biçimde ayrılmıştır; feature extraction ile classifier eğitimi birbirinden bağımsızdır.
- Train/validation/test rolleri karıştırılmaz; test seti yalnız final değerlendirmede kullanılır.
- Scaler model seçimi sırasında yalnız train üzerinde fit edilir.
- Exact-content SHA-256 kontrolü path isimlerinden bağımsız veri sızıntısı tespiti sağlar.
- Source-video bilgisi güvenilir değilse kod bunu uydurmaz ve video-level sonuç üretmez.
- Hashing uzun Drive işlemleri için retry, progress-save ve resume mekanizmasına sahiptir.
- SUCCESS statülü fakat okunamayan görüntü sessizce atlanmaz; audit üretilip süreç durur.
- Atomik model/CSV/JSON/YAML kayıt fonksiyonları yazma sırasında bozuk dosya riskini azaltır.
- Model kaydedildikten sonra fresh reload inference quality gate uygulanır.
- Run ID sayesinde deney çıktıları birbirinin üzerine yazılmaz.
- Environment, requirements, config ve manifest çıktıları deneyin daha sonra yeniden incelenmesini kolaylaştırır.

19. Teknik Sınırlamalar ve Dikkat Noktaları

- VGG16 tamamen dondurulmuştur; göz veri setine özel CNN fine-tuning yapılmaz.
- Görüntüler doğrudan 224x224 boyutuna resize edilir; aspect ratio koruyan padding/crop mekanizması yoktur.
- Notebook içinde veri augmentation uygulanmaz.

- SVM arama uzayı önceden belirlenmiş sınırlı bir grid'dir; otomatik geniş Bayesian/Optuna optimizasyonu yoktur.
- Final SVC probability=True kullanır; ayrı bir probability calibration aşaması tanımlanmamıştır.
- Video-level threshold 0.5 olarak sabittir.
- Source-video bilgisi eksikse video bazlı leakage ve video-level performans kesin olarak doğrulanamaz.
- Feature cache runlar arasında tekrar kullanılmadığı için VGG16 feature extraction her yeni run'da yeniden zaman alır.
- SHA-256 bütün veri üzerinde hesaplandığı için Drive ortamında uzun sürebilir; resume mekanizması bu maliyeti azaltmak için eklenmiştir.
- Bu notebook frame/ROI sınıflandırma pipeline'ıdır; temporal video modelleme veya blink/iris/corneal-reflection gibi ek biyolojik özellikler kullanmaz.

20. Özet Pseudocode

1. Gerekli paketleri kur; seed=42 ayarla.
2. Google Drive'ı bağla ve Deney 1 / Göz / eye_roi_output yolunu çöz.
3. Benzersiz RUN_ID oluştur ve run klasörlerini aç.
4. SSOT config dosyasını atomik olarak kaydet.
5. metadata.csv dosyasını oku; zorunlu sütunları doğrula.
6. Canonical metadata oluştur; source_sample_id'yi koru ve canonical sample_id üret.
7. Accounting equality, duplicate, split ve SUCCESS path kontrollerini yap.
8. Her ROI için SHA-256 hesapla; progress kaydet ve gerektiğinde resume et.
9. Aynı içerik farklı splitlerdeyse deneyi durdur.
10. source_video güvenilir ise train/val/test video kümelerini isdisjoint ile doğrula.
11. ImageNet pretrained VGG16'yı include_top=False, pooling=avg, trainable=False yükle.
12. İki batch forward + mini SVM smoke testini çalıştır.
13. Train/val/test ROI'lerini 224x224 RGB batch'ler halinde oku ve preprocess_input uygula.
14. VGG16 ile her görüntü için 512-D feature çıkar ve NPZ olarak sakla.
15. StandardScaler'i train üzerinde fit et; validation'a yalnız transform uygula.
16. Linear ve RBF SVM adaylarını train üzerinde fit et.
17. Validation F1 -> ROC-AUC -> Accuracy sıralamasıyla en iyi SVM'yi seç.
18. Train+validation feature'larını birleştir ve yeni scaler fit et.
19. Final SVC'yi train+validation üzerinde eğit.
20. Model/scaler/state dosyalarını atomik olarak kaydet ve fresh reload inference testini yap.
21. Test setinde class, probability ve decision score üret.
22. Frame-level metrikleri, classification report ve confusion matrix'i kaydet.
23. source_video doğrulanmışsa video-level olasılık ortalaması ve metrikleri hesapla.
24. Confusion Matrix, ROC ve Precision-Recall görsellerini üret ve kalite kontrolünden geçir.
25. Environment, requirements, metrics, run_summary ve output_manifest dosyalarını oluştur.
26. Zorunlu teslim dosyalarının tamamının mevcut olduğunu kontrol et.

Tek cümlelik özet

Notebook, combined-eye ROI görüntülerini önce dondurulmuş ImageNet VGG16 ile 512 boyutlu derin özelliklere dönüştürür; bu özellikleri train-only ölçekleme ve validation tabanlı hiperparametre seçimiyle SVM kullanarak REAL/FAKE sınıflandırır ve sürecin her aşamasını SSOT quality gate, atomik kayıt, SHA-256 leakage kontrolü ve tekrarlanabilirlik dosyalarıyla denetlenebilir hale getirir.